

目录

1 工业轴承故障预测系统的实现技术预研报告 1

1.1 文档信息 1

1.2 版本记录 1

1.3 1. 项目技术目标 2

1.4 2. 候选技术方案调研 2

1.4.1 2.1 开发语言候选 2

1.4.2 2.2 环境管理候选 2

1.5 3. Python 技术栈调研 2

1.5.1 3.1 数据处理类库 2

1.5.2 3.2 机器学习与深度学习类库 2

1.5.3 3.3 可视化与实验跟踪工具 3

1.6 4. uv 环境管理方案说明 3

1.7 5. 数据处理与特征工程技术选型 3

1.7.1 5.1 预处理候选方案 3

1.7.2 5.2 特征工程候选方案 3

1.8 6. 故障预测模型调研 3

1.8.1 6.1 传统模型 3

1.8.2 6.2 序列深度学习模型 4

1.8.3 6.3 退化分析与不确定性建模 4

1.9 7. 数据集来源与可用性分析 4

1.9.1 7.1 XJTU-SY 4

1.9.2 7.2 PHM2012 / FEMTO 4

1.9.3 7.3 数据集选型结论 4

1.10 8. 技术方案对比与最终选型 4

1.10.1 8.1 总体方案对比 4

1.10.2 8.2 最终选型结论 4

1.11 9. 预研结论 5

1 工业轴承故障预测系统的实现技术预研报告

1.1 文档信息

项目	内容
文档名称	技术预研报告
文档阶段	proposal
项目名称	工业轴承故障预测系统的实现
版本	V2.0
编写日期	2026-03-12
文档负责人	zdh
参与编写	zyj、zdh、zy
技术基线	Python 3.11+、uv、NumPy、Pandas、PyTorch、Scikit-learn

1.2 版本记录

版本	日期	编写人	说明
V1.0	2025-11-09	zdh、cyj、zyj	完成候选技术调研
V2.0	2026-03-12	zdh、cyj、zyj、zy	增补对比表、数据集分析和最终选型结论

1.3 1. 项目技术目标

技术预研的目的不是罗列工具，而是围绕项目目标回答三个问题：

1. 需要哪些技术才能支撑“真实数据接入 + 多模型训练 + 实验可复现”这一完整流程。
2. 候选方案之间有哪些差异，哪些更适合课程项目的进度和资源约束。
3. 如何在保证可运行的前提下，为后续扩展预留空间。

结合项目需求，技术预研的具体目标如下：

1. 构建稳定的 Python 开发环境和依赖管理方案。
2. 选择适合时序数据处理与模型训练的核心库。
3. 调研轴承故障预测常见模型，并确定课程项目可控的实现范围。
4. 评估 XJTU-SY 和 PHM2012 / FEMTO 数据集的可用性、获取方式和适配难度。
5. 给出最终技术栈与保留备选方案。

1.4 2. 候选技术方案调研

1.4.1 2.1 开发语言候选

方案	优点	缺点	结论
Python	数据科学生态成熟，开发速度快，适合课程项目	运行性能不如 C++	采用
Java	工程结构清晰，适合后端系统	科学计算生态不占优	不采用
MATLAB	信号处理能力强	商业授权限制，工程化和协作较弱	不采用

综合考虑课程周期、开源生态和团队技术基础，Python 是最合适的主语言。

1.4.2 2.2 环境管理候选

方案	优点	缺点	结论
uv	安装快、锁文件明确、统一管理虚拟环境和依赖	团队需统一使用方式	采用
pip + venv	通用性强	依赖解析和锁定能力弱	作为兼容方案
conda	包含较多科学计算包	环境较重，不利于课程提交	不采用

本项目统一使用 uv 管理依赖与虚拟环境，以保证开发、测试和答辩环境一致。

1.5 3. Python 技术栈调研

1.5.1 3.1 数据处理类库

库	适用场景	评估结果
NumPy	数值计算、数组运算、FFT	必选，作为基础数值层
Pandas	表格数据组织、日志和结果导出	必选，便于数据集抽象和结果分析
SciPy	频域分析和统计计算	可选，当前基础实现可不强依赖

1.5.2 3.2 机器学习与深度学习类库

库	适用场景	评估结果
Scikit-learn	传统模型、标准化、指标评估	必选，指标和部分基线模型依赖
PyTorch	CNN、RNN、Transformer 训练	必选，统一深度学习框架
XGBoost	树模型回归基线	可选，作为高级依赖保留

库	适用场景	评估结果
lifelines	生存分析建模	可选，作为生存分析增强方案保留

1.5.3 3.3 可视化与实验跟踪工具

工具	用途	评估结果
Matplotlib	基础图表输出	必选
Seaborn	统计图美化	必选
TensorBoard	训练曲线、梯度和参数可视化	必选

1.6 4. uv 环境管理方案说明

项目统一使用 uv，原因如下：

1. 依赖解析速度快，适合频繁调整环境的课程项目。
2. 能够通过 `pyproject.toml` 和 `uv.lock` 固定版本，降低“在我电脑能跑”的问题。
3. 便于区分基础依赖和高级可选依赖。

项目环境管理规则如下：

1. Python 版本统一为 3.11 及以上。
2. 基础依赖包含 NumPy、Pandas、Scikit-learn、PyTorch、Matplotlib、Seaborn、TensorBoard 等。
3. advanced 可选依赖包含 tsfresh、xgboost 等，用于增强实验。
4. dev 依赖包含 pytest 等测试工具。

1.7 5. 数据处理与特征工程技术选型

1.7.1 5.1 预处理候选方案

围绕轴承振动信号，本项目重点调研以下预处理能力：

1. 滑动窗口：将长序列切分为固定长度样本，是建模前提。
2. 归一化或标准化：降低不同工况下幅值差异对训练的影响。
3. 异常值和空值处理：提升真实数据导入的稳定性。
4. 多通道组织：同时支持水平振动、垂直振动和温度通道。

综合评估后，项目采用“默认轻量预处理 + 可扩展管道”的方案，即基础流程由自定义处理模块完成，可选接入更复杂的外部工具。

1.7.2 5.2 特征工程候选方案

方案	内容	优点	缺点	结论
手工统计特征	均值、标准差、RMS、峰值、峭度等	可解释性强、实现简单	依赖人工经验	采用
FFT 频域特征	主频、频域能量、谱质心等	适合振动场景	频率分辨率依赖窗口设置	采用
tsfresh 自动特征	大量自动化统计特征	覆盖广	依赖较重、特征筛选复杂	作为高级可选方案

最终选型为“基础统计特征 + 频域特征 + 可选自动特征”的混合策略。

1.8 6. 故障预测模型调研

1.8.1 6.1 传统模型

传统模型包括线性回归、随机森林、梯度提升树和 XGBoost。这些方法训练稳定、解释性较强，适合作为基线方案，但对于原始时序建模能力有限。项目保留这类方法作为参考基线，不作为主训练框架核心。

1.8.2 6.2 序列深度学习模型

1. CNN：擅长局部模式提取，适合对固定长度振动窗口进行建模。
2. RNN / LSTM：适合建模时间依赖关系，但训练速度相对较慢。
3. Transformer：适合处理中长序列和注意力分析，便于输出注意力热图，但计算代价较高。

综合考虑课程展示效果和工程扩展性，项目将 CNN、RNN 和 Transformer 都纳入实现范围。

1.8.3 6.3 退化分析与不确定性建模

除了直接 RUL 预测，项目还调研了以下扩展能力：

1. 3sigma 与 FPT 退化阶段划分，用于阶段分类与健康状态标注。
2. Monte Carlo Dropout 不确定性估计，用于输出均值和方差。
3. Kaplan-Meier / Cox 等生存分析方法，用于未来存活概率评估。

这些能力能够提升系统完整度，因此被纳入总体方案。

1.9 7. 数据集来源与可用性分析

1.9.1 7.1 XJTU-SY

XJTU-SY 适合研究轴承全寿命退化过程，优点是数据结构清晰、故障退化过程明确，适合做 RUL 和阶段分析。潜在风险在于下载入口可能发生变更，因此需要设计多入口兼容和本地目录加载能力。

1.9.2 7.2 PHM2012 / FEMTO

PHM2012 / FEMTO 是轴承寿命预测经典数据集，包含多工况数据，适合作为真实数据验证基准。需要注意的问题主要是压缩包体积大、目录层次较多，需要针对加速度与温度文件做统一抽象。

1.9.3 7.3 数据集选型结论

项目最终同时采用 XJTU-SY 和 PHM2012 / FEMTO：

1. 两者互补，可覆盖不同目录结构和工况组织方式。
2. 有利于验证数据加载器的通用性。
3. 有利于课程展示中说明系统不是只适配一个演示数据集。

1.10 8. 技术方案对比与最终选型

1.10.1 8.1 总体方案对比

维度	轻量脚本方案	单模型研究方案	本项目采用方案
数据集支持	单数据集	单或双数据集	多数据集统一抽象
预处理	固定脚本	固定流程	组件化管道
模型	单模型	少量模型	多模型可切换
训练过程监控	基本无	简单日志	TensorBoard + 回调
实验复现	较弱	中等	配置、历史、结果统一记录
软件工程文档	薄弱	一般	完整课程文档体系

1.10.2 8.2 最终选型结论

最终技术方案为：

1. Python 3.11+ 作为唯一开发语言。
2. uv 作为唯一环境与依赖管理工具。
3. NumPy、Pandas、Scikit-learn、PyTorch、Matplotlib、Seaborn、TensorBoard 作为核心基础栈。

4. XGBoost、tsfresh、lifelines 作为高级可选增强栈。
5. XJTU-SY 与 PHM2012 / FEMTO 作为主要真实数据集。

1.11 9. 预研结论

预研结论如下：

1. 技术栈成熟，可在课程周期内完成。
2. 数据集可获得，已具备真实数据验证条件。
3. 深度学习与传统方法可以在同一框架下共存，利于对比实验。
4. uv 可以有效提升环境一致性和可复现性，应作为统一规范写入后续所有文档。
5. 后续文档需要统一采用同一模块划分、同一术语和同一时间线，避免需求、设计和测试口径不一致。